

BÁO CÁO THỰC HÀNH

Thực hành An toàn mạng máy tính

Lab 3: Hash Functions and Digital Signatures

GVTH: Nguyễn Ngọc Trường

Nhóm 11 – NT101.Q22.1

THÔNG TIN CHUNG

MSSV	Họ và tên	Nội dung báo cáo
24520782	Lâm Mai Hồ Nhật Khánh	Câu 3 (tại lớp)
24520435	Lê Văn Anh Hải	Câu 4 (tại lớp)
24520613	Nguyễn Thái Hùng	Câu 2 (tại lớp)

MỤC LỤC:

Nhiệm vụ 2.2:	3
1. Xem xét hai thông điệp (message) dạng HEX như sau:	3
2. Xem xét hai chương trình thực thi có tên hello and erase: Chạy các chương trình này từ console và quan sát điều gì xảy ra. Hãy tạo giá trị băm MD5 cho các chương trình này và báo cáo quan sát của bạn.	4
3. Tải 2 tệp tin PDF: shattered-1.pdf và shattered-2.pdf. Mở các tệp này để kiểm tra sự khác biệt. Sau đó, tạo băm SHA-1 cho chúng và quan sát kết quả.	5
4. Rút ra kết luận dựa trên các quan sát của bạn, giải thích lý cho sự tồn tại của xung đột trong MD5 và SHA-1	6
Nhiệm vụ 2.3:	6
1. Mục tiêu	6
2. Cơ sở lý thuyết	6
2.1 Thuật toán MD5 và cấu trúc block	6
2.2 MD5 Collision và md5collgen	7
3. Thực hiện	7
3.1. Trường hợp 1: Tiền tố không phải bội số của 64 byte	7
3.2. Trường hợp 2: Tiền tố có đúng 64 byte	9
4. Trả lời câu hỏi	11
Câu 1: Nếu độ dài tiền tố không phải bội số của 64 byte thì điều gì xảy ra?	11
Câu 2: Khi tiền tố có đúng 64 byte thì điều gì xảy ra?	11
5. Kết luận	11
Nhiệm vụ 2.4:	11
Bước 1: Tải về một chứng chỉ từ Webserver thật	11
Bước 2: Trích xuất public key (e, n) từ chứng chỉ của nhà phát hành	12
Bước 3: Trích xuất Chữ ký từ chứng chỉ máy chủ	13
Bước 4: Trích xuất nội dung của chứng chỉ máy chủ	13

Nhiệm vụ 2.2

1. Xem xét hai thông điệp (message) dạng HEX như sau:

Message 1:

```
d131dd02c5e6eec4693d9a0698aff95c2fcab58712467eab4004583eb8fb7f8955ad340609f4b
30283e488832571415a085125e8f7cdc99fd91dbdf280373c5bd8823e3156348f5bae6dacd43
6c919c6dd53e2b487da03fd02396306d248cda0e99f33420f577
ee8ce54b67080a80d1ec69821bcb6a8839396f9652b6ff72a70
```

Message 2:

```
d131dd02c5e6eec4693d9a0698aff95c2fcab50712467eab4004583eb8fb7f8955ad340609f4b
30283e4888325f1415a085125e8f7cdc99fd91dbd7280373c5bd8823e3156348f5bae6dacd43
6c919c6dd53e23487da03fd02396306d248cda0e99f33420f577e
e8ce54b67080280d1ec69821bcb6a8839396f965ab6ff72a70
```

Có bao nhiêu byte khác biệt giữa hai thông điệp? Hãy tạo giá trị băm MD5 cho mỗi thông điệp và quan sát xem các giá trị MD5 này có giống nhau hay không và mô tả quan sát của bạn trong báo cáo lab

Các byte khác biệt giữa hai thông điệp:

- Số lượng byte khác biệt: Hai thông điệp có tổng cộng 6 vị trí khác nhau về ký tự Hex.
- Vị trí thay đổi: Các sự thay đổi này nằm rải rác ở cả 4 dòng dữ liệu

Các bước thực hiện:

- Chuyển đổi hai chuỗi HEX sang dạng dữ liệu nhị phân (bytes)
- Sử dụng hàm hashlib.md5 để tạo giá trị băm cho từng thông điệp
- so sánh hai kết quả mã băm thu được

```
import hashlib

msg1 = "d131dd02c5e6eec4693d9a0698aff95c2fca58712467eab4004583eb
msg2 = "d131dd02c5e6eec4693d9a0698aff95c2fca50712467eab4004583eb

hash1 = hashlib.md5(bytes.fromhex(msg1)).hexdigest()
hash2 = hashlib.md5(bytes.fromhex(msg2)).hexdigest()

print(hash1)
print(hash2)
```

79054025255fb1a26e4bc422aef54eb4
79054025255fb1a26e4bc422aef54eb4

Kết quả thực hiện:

- Message 1 và Message 2 là hai thông điệp khác nhau, nhưng hàm băm MD5 lại trả về cùng một giá trị duy nhất

2. Xem xét hai chương trình thực thi có tên hello and erase: Chạy các chương trình này từ console và quan sát điều gì xảy ra. Hãy tạo giá trị băm MD5 cho các chương trình này và báo cáo quan sát của bạn.

Mô tả thực hiện: Tải hai tệp thực thi hello và erase lên Colab, cấp quyền thực thi bằng lệnh `chmod +x` và tiến hành kiểm tra mã băm MD5

Các bước thực hiện:

- Tải hai tệp thực thi hello và erase lên môi trường Google Colab.
- Sử dụng lệnh `!chmod +x` để cấp quyền thực thi cho các tệp.
- Sử dụng lệnh `!file` để kiểm tra định dạng tệp và lệnh `!md5sum` để tính toán giá trị băm MD5.

```
!chmod +x /content/erase /content/hello

print('Kiểm tra tệp hello:')
!file /content/hello

print('\nKiểm tra tệp erase:')
!file /content/erase

print("\nKiểm tra mã băm MD5:")
!md5sum /content/hello /content/erase
```

... Kiểm tra loại tệp hello:
/content/hello: ELF 32-bit LSB executable, Intel 80386, version 1

Kiểm tra loại tệp erase:
/content/erase: ELF 32-bit LSB executable, Intel 80386, version 1

Kiểm tra mã băm MD5:
da5c61e1edc0f18337e46418e48c1290 /content/hello
da5c61e1edc0f18337e46418e48c1290 /content/erase

Kết quả thực hiện:

- Loại tệp: Cả hai tệp đều là định dạng thực thi ELF 32-bit LSB executable cho hệ điều hành Linux.
- Mã băm của hello: da5c61e1edc0f18337e46418e48c1290
- Mã băm của erase: da5c61e1edc0f18337e46418e48c1290
- Nhận xét: Hai tệp này hoàn toàn khác nhau về chức năng và nội dung nhưng lại có giá trị băm MD5 trùng khớp

3. Tải 2 tệp tin PDF: shattered-1.pdf và shattered-2.pdf. Mở các tệp này để kiểm tra sự khác biệt. Sau đó, tạo băm SHA-1 cho chúng và quan sát kết quả.

Mô tả thực hiện: Sử dụng công cụ sha1sum để kiểm tra tính kháng va chạm của thuật toán SHA-1 thông qua hai tệp tin PDF mẫu shattered-1.pdf và shattered-2.pdf.

```
!sha1sum shattered-1.pdf shattered-2.pdf
```

... 38762cf7f55934b34d179ae6a4c80cadccbb7f0a shattered-1.pdf
38762cf7f55934b34d179ae6a4c80cadccbb7f0a shattered-2.pdf

Kết quả thực hiện:

Mã băm SHA-1 của shattered-1.pdf: 38762cf7f55934b34d179ae6a4c80cadccbb7f0a

Mã băm SHA-1 của shattered-2.pdf: 38762cf7f55934b34d179ae6a4c80cadccbb7f0a

Nhận xét: Hai tệp tin có nội dung và cấu trúc dữ liệu khác nhau nhưng lại có chung một giá trị băm SHA-1

4. Rút ra kết luận dựa trên các quan sát của bạn, giải thích lý cho sự tồn tại của xung đột trong MD5 và SHA-1

Lý do tồn tại xung đột:

- Nguyên lý Chuồng bồ câu (Pigeonhole Principle): Vì không gian đầu vào (thông điệp) là vô hạn, trong khi không gian đầu ra (mã băm) là hữu hạn (128 bit cho MD5 và 160 bit cho SHA-1), nên về mặt lý thuyết, chắc chắn sẽ tồn tại ít nhất hai thông điệp khác nhau cho ra cùng một mã băm.
- Điểm yếu trong cấu trúc Merkle-Damgård: Cả MD5 và SHA-1 đều sử dụng cấu trúc Merkle-Damgård. Cấu trúc này chia dữ liệu thành các khối (blocks) và xử lý chúng tuần tự qua một hàm nén

Nhiệm vụ 2.3:

1. Mục tiêu

Nghiên cứu tấn công MD5 Collision — kỹ thuật tạo ra hai tệp khác nhau nhưng có cùng giá trị băm MD5. Cụ thể, thực hành tập trung vào:

- Sử dụng công cụ md5collgen để sinh hai tệp out1.bin và out2.bin từ cùng một tệp tiền tố (prefix).
- Quan sát hành vi của công cụ khi prefix không phải bội số của 64 byte.
- Quan sát hành vi khi prefix có đúng 64 byte (một block MD5 hoàn chỉnh).
- Xác minh rằng hai tệp đầu ra khác nội dung nhưng có cùng MD5 hash.

2. Cơ sở lý thuyết

2.1 Thuật toán MD5 và cấu trúc block

MD5 (Message Digest Algorithm 5) xử lý dữ liệu đầu vào theo từng block 512 bit (64 byte). Quá trình băm diễn ra như sau:

- Dữ liệu đầu vào được chia thành các block 64 byte.
- Nếu dữ liệu không chia hết cho 64, MD5 tự thêm padding (đệm) để hoàn thành block cuối.
- Mỗi block được xử lý tuần tự, cập nhật trạng thái nội (internal state) 128 bit.
- Giá trị hash cuối cùng là trạng thái nội sau khi xử lý hết tất cả block.

2.2 MD5 Collision và md5collgen

MD5 Collision là hiện tượng hai đầu vào khác nhau cho ra cùng một giá trị hash MD5. Công cụ md5collgen triển khai thuật toán tấn công của Wang et al. (2004) để tìm cặp collision trong thời gian thực tế.

Cơ chế hoạt động của md5collgen:

- Đọc tệp prefix đầu vào và padding lên bội số 64 nếu cần.
- Từ trạng thái MD5 sau khi xử lý prefix, tìm cặp block 128 byte (M, M') sao cho $\text{hash}(\text{prefix} \parallel M) = \text{hash}(\text{prefix} \parallel M')$.
- Xuất hai tệp: out1 = prefix + padding + M và out2 = prefix + padding + M'.

Kết quả: Hai tệp giống nhau hoàn toàn ở phần prefix + padding, nhưng khác nhau ở 128 byte collision block — và có cùng MD5 hash.

3. Thực hiện

3.1. Trường hợp 1: Tiền tố không phải bội số của 64 byte

Bước 1: Tạo tiền tố

```
echo "Lam Mai Ho Nhut Khanh" > prefix.txt
```

```
wc -c prefix.txt
```

```
nhutkhanh@nhutkhanh-Modern-15-A5M:~/md5collgen$ echo "Lam Mai Ho Nhut Khanh" > prefix.txt
```

Kết quả: độ dài không chia hết cho 64 byte.

Bước 2: Tạo và chạy

```
./md5collgen -p prefix.txt -o out1.bin out2.bin
```

```
nhutkhanh@nhutkhanh-Modern-15-A5M:~/md5collgen$ ./md5collgen -p prefix.txt -o out1.bin out2.bin
MD5 collision generator v1.5
by Marc Stevens (http://www.win.tue.nl/hashclash/)

Using output filenames: 'out1.bin' and 'out2.bin'
Using prefixfile: 'prefix.txt'
Using initial value: 02f3d09bc935462d606df678c8782116

Generating first block: .....
Generating second block: S10.....
Running time: 24.625583 s
```

Bước 3: Kiểm tra

```
diff out1.bin out2.bin
```

Lab 3: Hash Functions and Digital Signatures

md5sum out1.bin

md5sum out2.bin

```
nhutkhanh@nhutkhanh-Modern-15-ASM:~/md5collgen$ echo "Lam Mai Ho Nhut Khanh" > prefix.txt
nhutkhanh@nhutkhanh-Modern-15-ASM:~/md5collgen$ ./md5collgen -p prefix.txt -o out1.bin out2.bin
MD5 collision generator v1.5
by Marc Stevens (http://www.win.tue.nl/hashclash/)

Using output filenames: 'out1.bin' and 'out2.bin'
Using prefixfile: 'prefix.txt'
Using initial value: 02f3d09bc935462d606df678c8782116

Generating first block: .....
.....
Generating second block: S10.....
Running time: 24.625583 s
nhutkhanh@nhutkhanh-Modern-15-ASM:~/md5collgen$ diff out1.bin out2.bin
Binary files out1.bin and out2.bin differ
nhutkhanh@nhutkhanh-Modern-15-ASM:~/md5collgen$ md5sum out1.bin
5ca8ab48c87d8af91b7c5cdef6737b7a out1.bin
nhutkhanh@nhutkhanh-Modern-15-ASM:~/md5collgen$ md5sum out2.bin
5ca8ab48c87d8af91b7c5cdef6737b7a out2.bin
nhutkhanh@nhutkhanh-Modern-15-ASM:~/md5collgen$
```

- 5ca8ab48c87d8af91b7c5cdef6737b7a out1.bin
- 5ca8ab48c87d8af91b7c5cdef6737b7a out2.bin

→ Hai giá trị hash GIỐNG NHAU

```
nhutkhanh@nhutkhanh-Modern-15-ASM:~/md5collgen$ xxd out1.bin
00000000: 4c61 6d20 4d61 6920 486f 204e 6875 7420  Lam Mai Ho Nhut
00000010: 4b68 616e 680a 0000 0000 0000 0000 0000  Khanh.....
00000020: 0000 0000 0000 0000 0000 0000 0000 0000  .....
00000030: 0000 0000 0000 0000 0000 0000 0000 0000  .....
00000040: a144 c42a 9ad0 8093 ef3c de6e be1a 2dce  .D.*.....<.n...
00000050: 5559 59b3 5dab 8b41 4f6e 415a f3a3 96ba  UYY.]..A0nAZ...
00000060: ccfc 01bf 7cab a113 8b0e 73f3 9c97 899d  ....|.....s....
00000070: 9d79 41d2 e894 1ce2 572a f561 f12a c460  .yA.....W*.a.*.
00000080: f7f2 287d f2b6 a616 bd66 7e7d 09e3 a08b  ..{).....f~}....
00000090: c373 4c20 68da 2110 0bc3 10df ab5f 88b6  .sL.h.!....._...
000000a0: cb13 7028 af49 7c5c f604 c83a 0408 ef22  ..p(.I|\....."
000000b0: d4b6 ace1 0695 9f27 4c19 75cf 2558 c102  ....'L.u.%X...
nhutkhanh@nhutkhanh-Modern-15-ASM:~/md5collgen$ xxd out2.bin
00000000: 4c61 6d20 4d61 6920 486f 204e 6875 7420  Lam Mai Ho Nhut
00000010: 4b68 616e 680a 0000 0000 0000 0000 0000  Khanh.....
00000020: 0000 0000 0000 0000 0000 0000 0000 0000  .....
00000030: 0000 0000 0000 0000 0000 0000 0000 0000  .....
00000040: a144 c42a 9ad0 8093 ef3c de6e be1a 2dce  .D.*.....<.n...
00000050: 5559 5933 5dab 8b41 4f6e 415a f3a3 96ba  UYY3]..A0nAZ...
00000060: ccfc 01bf 7cab a113 8b0e 73f3 9c17 8a9d  ....|.....s....
00000070: 9d79 41d2 e894 1ce2 572a f5e1 f12a c460  .yA.....W*...*.
00000080: f7f2 287d f2b6 a616 bd66 7e7d 09e3 a08b  ..{).....f~}....
00000090: c373 4ca0 68da 2110 0bc3 10df ab5f 88b6  .sL.h.!....._...
000000a0: cb13 7028 af49 7c5c f604 c83a 0488 ee22  ..p(.I|\....."
000000b0: d4b6 ace1 0695 9f27 4c19 754f 2558 c102  ....'L.u0%X...
nhutkhanh@nhutkhanh-Modern-15-ASM:~/md5collgen$
nhutkhanh@nhutkhanh-Modern-15-ASM:~/md5collgen$ diff out1.bin out2.bin
Binary files out1.bin and out2.bin differ
nhutkhanh@nhutkhanh-Modern-15-ASM:~/md5collgen$
```

Quan sát bằng công cụ xxd cho thấy:

- Phần tiền tố ban đầu giống nhau hoàn toàn.
- Các byte phía sau có sự khác biệt, tạo thành collision block.

Kết quả:

- Hai tệp có nội dung khác nhau
- Hai tệp có cùng giá trị băm MD5

3.2. Trường hợp 2: Tiền tố có đúng 64 byte

Bước 1: Tạo tiền tố 64 byte

printf

```
"AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA  
AAAAAAAAAAAAAAAA" > prefix64.txt
```

wc -c prefix64.txt

```
nhutkhanh@nhutkhanh-Modern-15-ASM:~/md5collgen$ printf "AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA  
.txt  
nhutkhanh@nhutkhanh-Modern-15-ASM:~/md5collgen$ wc -c prefix64.txt  
64 prefix64.txt
```

- Kết quả: 64 byte.

Bước 2: Tạo va chạm

```
./md5collgen -p prefix64.txt -o out1_64.bin out2_64.bin
```

```
nhutkhanh@nhutkhanh-Modern-15-ASM:~/md5collgen$ ./md5collgen -p prefix64.txt -o out1_64.bin out2_64.bin  
MD5 collision generator v1.5  
by Marc Stevens (http://www.win.tue.nl/hashclash/)  
  
Using output filenames: 'out1_64.bin' and 'out2_64.bin'  
Using prefixfile: 'prefix64.txt'  
Using initial value: b217e7185a63fe5f643fe6a6d401bf59  
  
Generating first block: .....  
Generating second block: S11.....  
Running time: 1.406816 s
```

Bước 3: Kiểm tra

```
md5sum out1_64.bin
```

```
md5sum out2_64.bin
```

```
diff out1_64.bin out2_64.bin
```

Lab 3: Hash Functions and Digital Signatures

```
nhutkhanh@nhutkhanh-Modern-15-ASM:~/md5collgen$ diff out1_64.bin out2_64.bin
Binary files out1_64.bin and out2_64.bin differ
nhutkhanh@nhutkhanh-Modern-15-ASM:~/md5collgen$ md5sum out1_64.bin
38eaa7076dc8cf9535da9e8c71c10cfc  out1_64.bin
nhutkhanh@nhutkhanh-Modern-15-ASM:~/md5collgen$ md5sum out2_64.bin
38eaa7076dc8cf9535da9e8c71c10cfc  out2_64.bin
nhutkhanh@nhutkhanh-Modern-15-ASM:~/md5collgen$ diff out1_64.bin out2_64.bin
Binary files out1_64.bin and out2_64.bin differ
nhutkhanh@nhutkhanh-Modern-15-ASM:~/md5collgen$ xxd out1_64.bin
00000000: 4141 4141 4141 4141 4141 4141 4141 4141  AAAAAAAAAAAAAAAAAA
00000010: 4141 4141 4141 4141 4141 4141 4141 4141  AAAAAAAAAAAAAAAAAA
00000020: 4141 4141 4141 4141 4141 4141 4141 4141  AAAAAAAAAAAAAAAAAA
00000030: 4141 4141 4141 4141 4141 4141 4141 4141  AAAAAAAAAAAAAAAAAA
00000040: 84b2 cfd8 2fe6 25c2 4344 6ac5 b882 d3fc  ....%.CDj.....
00000050: 5eaf b719 1faa 662b 1b7e 7931 d1a9 88fe  ^.....f+..y1....
00000060: 3cb3 03c8 f2ae 1418 0f68 ce33 f0df 8e4c  <.....h.3...L
00000070: 9a9a e237 bfdc 35d5 64cf 1ce2 dd63 8ed7  ...7..5.d..b.c..
00000080: 80b4 3bc5 3820 be08 12ea dd03 2adb bba4  ...;.8 .....*...
00000090: 8f5d e382 52cc 0c2b 7ed7 7fc7 5b4c 4dc1  .]..R..+...[LM.
000000a0: eacc b0ea 84a9 9257 19a5 6f99 85f3 001f  .....W..o.....
000000b0: 7c4f f389 dc49 3380 e391 7c7e 20e8 ad10  |0...I3...|~ ...
nhutkhanh@nhutkhanh-Modern-15-ASM:~/md5collgen$ xxd out2_64.bin
00000000: 4141 4141 4141 4141 4141 4141 4141 4141  AAAAAAAAAAAAAAAAAA
00000010: 4141 4141 4141 4141 4141 4141 4141 4141  AAAAAAAAAAAAAAAAAA
00000020: 4141 4141 4141 4141 4141 4141 4141 4141  AAAAAAAAAAAAAAAAAA
00000030: 4141 4141 4141 4141 4141 4141 4141 4141  AAAAAAAAAAAAAAAAAA
00000040: 84b2 cfd8 2fe6 25c2 4344 6ac5 b882 d3fc  ....%.CDj.....
00000050: 5eaf b799 1faa 662b 1b7e 7931 d1a9 88fe  ^.....f+..y1....
00000060: 3cb3 03c8 f2ae 1418 0f68 ce33 f05f 8f4c  <.....h.3...L
00000070: 9a9a e237 bfdc 35d5 64cf 1ce2 dd63 8ed7  ...7..5.d..c...
00000080: 80b4 3bc5 3820 be08 12ea dd03 2adb bba4  ...;.8 .....*...
00000090: 8f5d e302 52cc 0c2b 7ed7 7fc7 5b4c 4dc1  .]..R..+...[LM.
000000a0: eacc b0ea 84a9 9257 19a5 6f99 8573 001f  .....W..o..s...
000000b0: 7c4f f389 dc49 3380 e391 7c7e 20e8 ad10  |0...I3...|~ ...
nhutkhanh@nhutkhanh-Modern-15-ASM:~/md5collgen$
```

Quan sát kết quả cho thấy:

- 64 byte đầu tiên (tiền tố) giống nhau hoàn toàn.
- Các byte phía sau có sự khác biệt tại nhiều vị trí.
- md5sum out1_64.bin: 38eaa7076dc8cf9535da9e8c71c10cfc out1_64.bin
- md5sum out2_64.bin: 38eaa7076dc8cf9535da9e8c71c10cfc out2_64.bin

→ Hai giá trị hash GIỐNG NHAU

- Nội dung khác nhau
- Giá trị MD5 giống nhau

Kết quả thực nghiệm

Sau khi thực hiện, thu được hai tệp out1_64.bin và out2_64.bin.

- Kết quả lệnh diff cho thấy hai tệp khác nhau về nội dung.
- Tuy nhiên, giá trị băm MD5 của hai tệp là hoàn toàn giống nhau.

Điều này chứng minh rằng đã xảy ra va chạm MD5.

Quan sát bằng công cụ xxd cho thấy:

- 64 byte đầu tiên (tiền tố) giống nhau hoàn toàn.
- Các byte phía sau có sự khác biệt tại nhiều vị trí.
- Sự khác biệt này được thiết kế đặc biệt để tạo ra cùng giá trị băm.

4. Trả lời câu hỏi

Câu 1: *Nếu độ dài tiền tố không phải bội số của 64 byte thì điều gì xảy ra?*

Trả lời: Thuật toán MD5 xử lý dữ liệu theo từng khối 512 bit (64 byte). Khi tiền tố không phải bội số của 64 byte, chương trình sẽ tự động thêm padding để hoàn chỉnh khối dữ liệu. Va chạm vẫn được tạo ra nhưng không nằm đúng ranh giới khối, khiến cấu trúc dữ liệu khó kiểm soát hơn.

Câu 2: *Khi tiền tố có đúng 64 byte thì điều gì xảy ra?*

Trả lời: Khi tiền tố có độ dài đúng 64 byte, nó tương ứng chính xác với một khối xử lý của MD5. Do đó, va chạm sẽ được tạo ra ở khối tiếp theo mà không cần thêm padding. Điều này giúp quá trình tạo va chạm rõ ràng và dễ kiểm soát hơn.

5. Kết luận

Bài thực hành cho thấy có thể tạo ra hai tệp khác nhau nhưng có cùng giá trị băm MD5. Điều này chứng minh rằng MD5 không còn đảm bảo tính toàn vẹn dữ liệu và không an toàn trong các ứng dụng bảo mật hiện nay. Việc sử dụng các thuật toán mạnh hơn như SHA-256 là cần thiết.

Nhiệm vụ 2.4:

Thực hiện lại các bước trên để trích xuất tất cả thông tin, bao gồm: khóa công khai của CA, chữ ký của CA và phần thân của chứng chỉ máy chủ. Viết chương trình Python để xác minh chứng chỉ dựa trên công thức RSA.

Trả lời:

Thực hiện lại các bước trên với máy chủ web google.com:

Bước 1: Tải về một chứng chỉ từ Webserver thật.

```
openssl s_client -connect www.google.com:443 -showcerts
```

Lab 3: Hash Functions and Digital Signatures

```
halmoi1@ubuntu:~$ openssl s_client -connect www.google.com:443 -showcerts
CONNECTED(00000003)
depth=2 C = US, O = Google Trust Services LLC, CN = GTS Root R1
verify return:1
depth=1 C = US, O = Google Trust Services, CN = WR2
verify return:1
depth=0 CN = www.google.com
verify return:1
---
Certificate chain
 0 s:CN = www.google.com
  i:C = US, O = Google Trust Services, CN = WR2
-----BEGIN CERTIFICATE-----
MIIEVjCCAz6gAwIBAgIRAK4lRa9f1MxRCYHXk1amy0EwDQYJKoZIhvcNAQELBQAw
OzELMAkGA1UEBhMCVVMxHjAcBgNVBAoTFUdvdv2dsZS8UcnVzdCBTZXJ2aWNlc2EM
MAoGA1UEAxMDV1IyMB4XDTI2MDMzMzA4Mzc2NloXDTI2MDYyMjA4Mzc2NVowGTEX
MBUGA1UEAxM0d3d3Lmdvb2dsZS5jb20wWTATBgqhkhjOPQIBggqhkhjOPQMBBwNC
AATHie5U58E2V7y37sKS7d+KbDdcMxQYR7070DhC63zi+mLuhtGvIUB4vrT5Z5NW
OQFbgrmt4VdaYQBQcag4X1Xyo4ICQDCCAajwwDgYDVVR0PAQH/BAQDAgeAMBMA1Ud
JQQMAAoGCCsGAQUFBwMBMAwGA1UdEwEB/wQCMAAwHQYDVVR0B0BBYEFASXNMLsOWM
3fMpGuyx+FR9meAwMB8GA1UdIwQYMBaAFN4bHu15FdQ+NyTDIbvsNDltQrIwMFgg
```

Bước 2: Trích xuất public key (e, n) từ chứng chỉ của nhà phát hành

Lấy modulus (n):

```
openssl x509 -in c1.pem -noout -modulus
```

```
halmoi1@ubuntu:~/NT101/Lab03$ openssl x509 -in c1.pem -noout -modulus
Modulus=A9FF9C7F451E70A8539FCAD9E50DDE4657577DBC8F9A5AAC46F1849ABB91DBC9FB2F01FB920900165EA01CF8C1ABF9782F4ACCD885A2D
8593C0ED318FBB1F5240D26EEB65B64767C14C72F7ACEA84CB7F4D908FCD8F7233520A8E269E28C4E3FB159FA60A21EB3C920531982CA36536D60
4DE90091FC768D5C080F0AC2DCF1736BC5136E0A4F7AC2F2021C2EB46383DA31F62D7530B2FBABC26EDBA9C00EB9F967D4C3255774EB05B4E98EB
5DE28DCDC7A14E47103CB4D612E6157C519A90B98841AE87929D9B28D2FF576A66E0CEAB95A82996637012671E3AE1DDB02171D77C9EFDA176E
FE2BFB381714D166A7AF9AB570CCC863813A8CC02AA97637CEE3
```

Lấy e:

```
openssl x509 -in c1.pem -text -noout
```

```
halmoi1@ubuntu:~/NT101/Lab03$ openssl x509 -in c1.pem -text -noout
Certificate:
  Data:
    Version: 3 (0x2)
    Serial Number:
      7f:f0:05:a0:7c:4c:de:d1:00:ad:9d:66:a5:10:7b:98
    Signature Algorithm: sha256WithRSAEncryption
    Issuer: C = US, O = Google Trust Services LLC, CN = GTS Root R1
    Validity
      Not Before: Dec 13 09:00:00 2023 GMT
      Not After : Feb 20 14:00:00 2029 GMT
    Subject: C = US, O = Google Trust Services, CN = WR2
    Subject Public Key Info:
      Public Key Algorithm: rsaEncryption
      RSA Public-Key: (2048 bit)
      Modulus:
        00:a9:ff:9c:7f:45:1e:70:a8:53:9f:ca:d9:e5:0d:
        de:46:57:57:7d:bc:8f:9a:5a:ac:46:f1:84:9a:bb:
        91:db:c9:fb:2f:01:fb:92:09:00:16:5e:a0:1c:f8:
        c1:ab:f9:78:2f:4a:cc:d8:85:a2:d8:59:3c:0e:d3:
        18:fb:b1:f5:24:0d:26:ee:b6:5b:64:76:7c:14:c7:
        2f:7a:ce:a8:4c:b7:f4:d9:08:fc:df:87:23:35:20:
        a8:e2:69:e2:8c:4e:3f:b1:59:fa:60:a2:1e:b3:c9:
        20:53:19:82:ca:36:53:6d:60:4d:e9:00:91:fc:76:
        8d:5c:08:0f:0a:c2:dc:f1:73:6b:c5:13:6e:0a:4f:
```

Bước 3: Trích xuất Chữ ký từ chứng chỉ máy chủ.

Từ hình 3, tìm dòng Signature Algorithm, ngay dưới đó là khối Signature, một loạt các giá trị hex có dấu hai chấm. Sao chép các giá trị hex đó vào file signature_hex.txt.

Lab 3: Hash Functions and Digital Signatures

```
c0.pem X signature_hex.txt c1.pem
signature_hex.txt
1 00:a9:ff:9c:7f:45:1e:70:a8:53:9f:ca:d9:e5:0d:
2 1e:46:57:57:7d:bc:8f:9a:5a:ac:46:f1:84:9a:bb:
3 91:db:c9:fb:2f:01:fb:92:09:00:16:5e:a0:1c:f8:
4 c1:ab:f9:78:2f:4a:cc:d8:85:a2:d8:59:3c:0e:d3:
5 18:fb:b1:f5:24:0d:26:ee:b6:5b:64:76:7c:14:c7:
6 2f:7a:ce:a8:4c:b7:f4:d9:08:fc:df:87:23:35:20:
7 a8:e2:69:e2:8c:4e:3f:b1:59:fa:60:a2:1e:b3:c9:
8 20:53:19:82:ca:36:53:6d:60:4d:e9:00:91:fc:76:
9 8d:5c:08:0f:0a:c2:dc:f1:73:6b:c5:13:6e:0a:4f:
10 7a:c2:f2:02:1c:2e:b4:63:83:da:31:f6:2d:75:30:
11 b2:fb:ab:c2:6e:db:a9:c0:0e:b9:f9:67:d4:c3:25:
12 57:74:eb:05:b4:e9:8e:b5:de:28:cd:cc:7a:14:e4:
13 71:03:cb:4d:61:2e:61:57:c5:19:a9:0b:98:84:1a:
14 e8:79:29:d9:b2:8d:2f:ff:57:6a:66:e0:ce:ab:95:
15 a8:29:96:63:70:12:67:1e:3a:e1:db:b0:21:71:d7:
16 7c:9e:fd:aa:17:6e:fe:2b:fb:38:17:14:d1:66:a7:
17 af:9a:b5:70:cc:c8:63:81:3a:8c:c0:2a:a9:76:37:
18 ce:e3
```

Bước 4: Trích xuất nội dung của chứng chỉ máy chủ

Trích xuất phần thân từ offset 4 và lưu nó vào một tệp nhị phân:

```
openssl asn1parse -i -in c0.pem -strparse 4 -out c0_body.bin -noout
```

Viết chương trình Python để xác minh chứng chỉ dựa trên công thức RSA:

```
import hashlib

n_hex =
"A9FF9C7F451E70A8539FCAD9E50DDE4657577DBC8F9A5AAC46F1849ABB91DBC9FB2F01FB92
0900165EA01CF8C1ABF9782F4ACCD885A2D8593C0ED318FBB1F5240D26EEB65B64767C14C72F
7ACEA84CB7F4D908FCDF87233520A8E269E28C4E3FB159FA60A21EB3C920531982CA36536D604
DE90091FC768D5C080F0AC2DCF1736BC5136E0A4F7AC2F2021C2EB46383DA31F62D7530B2FB
ABC26EDBA9C00EB9F967D4C3255774EB05B4E98EB5DE28CDCC7A14E47103CB4D612E6157C51
9A90B98841AE87929D9B28D2FFF576A66E0CEAB95A82996637012671E3AE1DBB02171D77C9EF
DAA176EFE2BFB381714D166A7AF9AB570CCC863813A8CC02AA97637CEE3"

e = 65537

signature_hex =
"00a9ff9c7f451e70a8539fcad9e50dde4657577dbc8f9a5aac46f1849abb91dbc9fb2f01fb920900165ea01cf8
c1abf9782f4accd885a2d8593c0ed318fbb1f5240d26eeb65b64767c14c72f7acea84cb7f4d908fcd87233520
a8e269e28c4e3fb159fa60a21eb3c920531982ca36536d604de90091fc768d5c080f0ac2dcf1736bc5136e0a4
f7ac2f2021c2eb46383da31f62d7530b2fbabc26edba9c00eb9f967d4c3255774eb05b4e98eb5de28cdcc7a14
e47103cb4d612e6157c519a90b98841ae87929d9b28d2fff576a66e0ceab95a82996637012671e3ae1dbb021
```

Lab 3: Hash Functions and Digital Signatures

```
71d77c9efdaa176efe2bfb381714d166a7af9ab570ccc863813a8cc02aa97637cee3"

body_file = "c0_body.bin"

def verify_certificate():
    print("--- ĐANG BẮT ĐẦU QUÁ TRÌNH XÁC MINH ---")

    n = int(n_hex, 16)
    s = int(signature_hex, 16)

    decrypted_int = pow(s, e, n)
    decrypted_hex = hex(decrypted_int)[2:]

    print(f'[1] Dữ liệu giải mã từ chữ ký (chứa Padding + Hash):\n{decrypted_hex}\n')

    try:
        with open(body_file, "rb") as f:
            body_data = f.read()

            recomputed_hash = hashlib.sha256(body_data).hexdigest()
            print(f'[2] Mã băm SHA-256 tính toán từ {body_file}:\n{recomputed_hash}\n')
    except FileNotFoundError:
        print(f'Lỗi: Không tìm thấy tệp {body_file}. Hãy chắc chắn bạn đã chạy lệnh asn1parse.")
        return

    if recomputed_hash in decrypted_hex:
        print("=> KẾT LUẬN: CHÚNG CHỈ HỢP LỆ!")
    else:
        print("=> KẾT LUẬN: XÁC MINH THẤT BẠI!")

if __name__ == "__main__":
    verify_certificate()
```

Chạy chương trình:

```
● halmoil@ubuntu:~/NT101/Lab03$ python3 /home/halmoil/NT101/Lab03/rsa.py
--- ĐANG BẮT ĐẦU QUÁ TRÌNH XÁC MINH ---
[1] Dữ liệu giải mã từ chữ ký (chứa Padding + Hash):
0

[2] Mã băm SHA-256 tính toán từ c0_body.bin:
65afad13c5b8cafe313e174d5c41f2ad989a939ad79dd6cbb6bc6841583c7a03

=> KẾT LUẬN: XÁC MINH THẤT BẠI!
```

HẾT